Simple Hero API with FastAPI

Let's start by building a simple hero web API with **FastAPI**. ✨

Install FastAPI

The first step is to install FastAPI.

FastAPI is the framework to create the **web API**.

Make sure you create a virtual environment, activate it, and then install them, for example with:

```bash
fast →

$ pip insta
```

SQLModel Code - Models, Engine

Now let's start with the SQLModel code.

We will start with the **simplest version**, with just heroes (no teams yet).

This is almost the same code we have seen up to now in previous examples:

Python 3.10+

```python
# Code above omitted 👆

from sqlmodel import Field, Session, SQLModel, create_engine, select

# Code here omitted 👈

class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)
```

```python
def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from fastapi import FastAPI
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/")
def create_hero(hero: Hero):
    with Session(engine) as session:
        session.add(hero)
        session.commit()
        session.refresh(hero)
        return hero


@app.get("/heroes/")
def read_heroes():
    with Session(engine) as session:
        heroes = session.exec(select(Hero)).all()
        return heroes
```

There's only one change here from the code we have used before, the `check_same_thread` in the `connect_args`.

That is a configuration that SQLAlchemy passes to the low-level library in charge of communicating with the database.

`check_same_thread` is by default set to `True`, to prevent misuses in some simple cases.

But here we will make sure we don't share the same **session** in more than one request, and that's the actual **safest way** to prevent any of the problems that configuration is there for.

And we also need to disable it because in **FastAPI** each request could be handled by multiple interacting threads.

Info

That's enough information for now, you can read more about it in the FastAPI docs for `async` and `await`.

The main point is, by ensuring you **don't share** the same **session** with more than one request, the code is already safe.

FastAPI App

The next step is to create the **FastAPI** app.

We will import the `FastAPI` class from `fastapi`.

And then create an `app` object that is an instance of that `FastAPI` class:

Python 3.10+

```
from fastapi import FastAPI
from sqlmodel import Field, Session, SQLModel, create_engine, select

# Code here omitted 👈

app = FastAPI()

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from fastapi import FastAPI
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/")
def create_hero(hero: Hero):
    with Session(engine) as session:
        session.add(hero)
        session.commit()
        session.refresh(hero)
        return hero


@app.get("/heroes/")
def read_heroes():
    with Session(engine) as session:
        heroes = session.exec(select(Hero)).all()
        return heroes
```

Create Database and Tables on `startup`

We want to make sure that once the app starts running, the function `create_db_and_tables` is called. To create the database and tables.

This should be called only once at startup, not before every request, so we put it in the function to handle the `"startup"` event:

Python 3.10+

```python
# Code above omitted 👆

app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()
```

```
# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from fastapi import FastAPI
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/")
def create_hero(hero: Hero):
    with Session(engine) as session:
        session.add(hero)
        session.commit()
        session.refresh(hero)
        return hero


@app.get("/heroes/")
def read_heroes():
    with Session(engine) as session:
        heroes = session.exec(select(Hero)).all()
        return heroes
```

Create Heroes *Path Operation*

Info

If you need a refresher on what a **Path Operation** is (an endpoint with a specific HTTP Operation) and how to work with it in FastAPI, check out the FastAPI First Steps docs.

Let's create the **path operation** code to create a new hero.

It will be called when a user sends a request with a `POST` **operation** to the `/heroes/` **path**:

Python 3.10+

```python
# Code above omitted 👆

app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/")
def create_hero(hero: Hero):
    with Session(engine) as session:
        session.add(hero)
        session.commit()
        session.refresh(hero)
        return hero

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from fastapi import FastAPI
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/")
def create_hero(hero: Hero):
    with Session(engine) as session:
        session.add(hero)
        session.commit()
        session.refresh(hero)
        return hero


@app.get("/heroes/")
def read_heroes():
    with Session(engine) as session:
        heroes = session.exec(select(Hero)).all()
        return heroes
```

Info

If you need a refresher on some of those concepts, checkout the FastAPI documentation:

- First Steps
- Path Parameters - Data Validation and Data Conversion
- Request Body

The **SQLModel** Advantage

Here's where having our **SQLModel** class models be both **SQLAlchemy** models and **Pydantic** models at the same time shine. ✨

Here we use the **same** class model to define the **request body** that will be received by our API.

Because **FastAPI** is based on Pydantic, it will use the same model (the Pydantic part) to do automatic data validation and conversion from the JSON request to an object that is an actual instance of the `Hero` class.

And then, because this same **SQLModel** object is not only a **Pydantic** model instance but also a **SQLAlchemy** model instance, we can use it directly in a **session** to create the row in the database.

So we can use intuitive standard Python **type annotations**, and we don't have to duplicate a lot of the code for the database models and the API data models. 🎉

> **Tip**
>
> We will improve this further later, but for now, it already shows the power of having **SQLModel** classes be both **SQLAlchemy** models and **Pydantic** models at the same time.

Read Heroes *Path Operation*

Now let's add another **path operation** to read all the heroes:

Python 3.10+

```python
# Code above omitted 👆

app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/")
def create_hero(hero: Hero):
    with Session(engine) as session:
        session.add(hero)
        session.commit()
        session.refresh(hero)
        return hero


@app.get("/heroes/")
def read_heroes():
    with Session(engine) as session:
        heroes = session.exec(select(Hero)).all()
        return heroes
```

👀 **Full file preview**

Python 3.10+

```python
from fastapi import FastAPI
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/")
def create_hero(hero: Hero):
    with Session(engine) as session:
        session.add(hero)
        session.commit()
        session.refresh(hero)
        return hero


@app.get("/heroes/")
def read_heroes():
    with Session(engine) as session:
        heroes = session.exec(select(Hero)).all()
        return heroes
```

This is pretty straightforward.

When a client sends a request to the **path** `/heroes/` with a `GET` HTTP **operation**, we run this function that gets the heroes from the database and returns them.

One Session per Request

Remember that we should use a SQLModel **session** per each group of operations and if we need other unrelated operations we should use a different session?

Here it is much more obvious.

We should normally have **one session per request** in most of the cases.

In some isolated cases, we would want to have new sessions inside, so, **more than one session** per request.

But we would **never want to *share* the same session** among different requests.

In this simple example, we just create the new sessions manually in the **path operation functions**.

In future examples later we will use a FastAPI Dependency to get the **session**, being able to share it with other dependencies and being able to replace it during testing. 🤓

Run the FastAPI Server in Development Mode

Now we are ready to run the FastAPI application.

Put all that code in a file called `main.py`.

Then run it with the `fastapi` CLI, in development mode:

```bash
```

> **Info**
>
> The `fastapi` command uses Uvicorn underneath.

When you use `fastapi dev` it starts Uvicorn with the option to reload automatically every time you make a change to the code, this way you will be able to develop faster. 🤓

Run the FastAPI Server in Production Mode

The development mode should not be used in production, as it includes automatic reload by default it consumes much more resources than necessary, and it would be more error prone, etc.

For production, use `fastapi run` instead of `fastapi dev`:

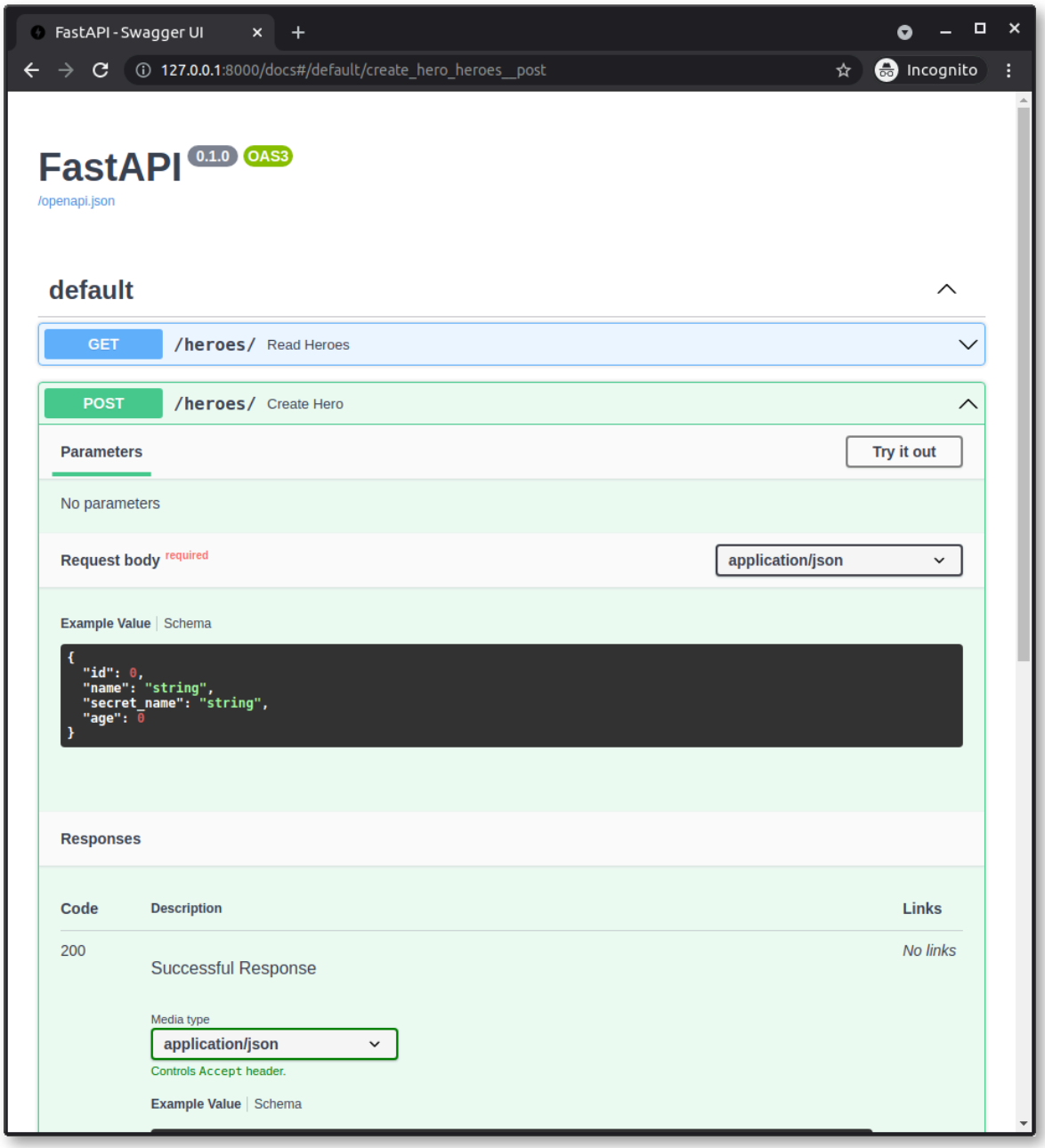
```bash
```

Check the API docs UI

Now you can go to that URL in your browser `http://127.0.0.1:8000`. We didn't create a *path operation* for the root path `/`, so that URL alone will only show a "Not Found" error... that "Not Found" error is produced by your FastAPI application.

But you can go to the **automatically generated interactive API documentation** at the path `/docs`: http://127.0.0.1:8000/docs. ✨

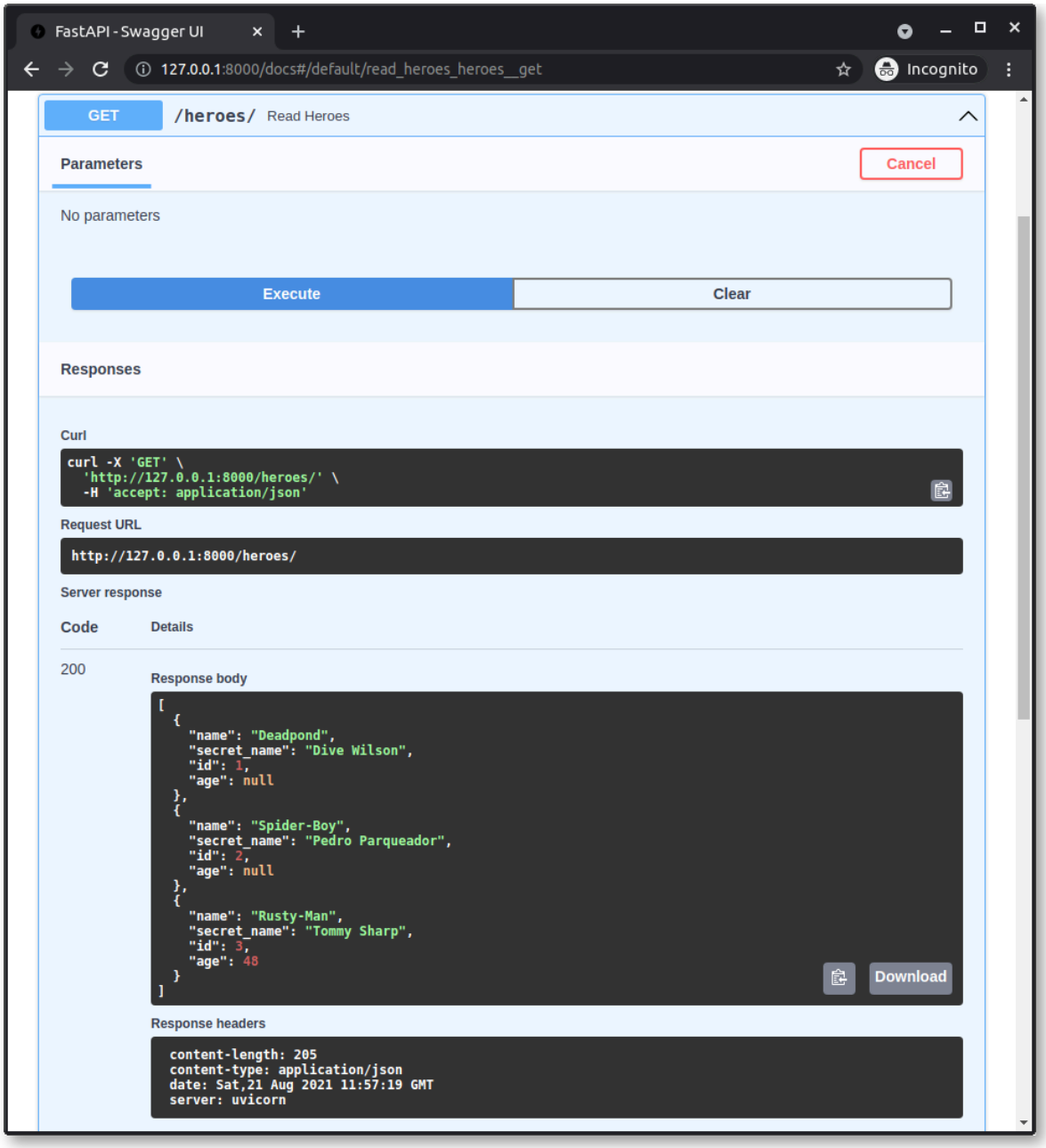
You will see that this **automatic API docs UI** has the *paths* that we defined above with their *operations*, and that it already knows the shape of the data that the **path operations** will receive:



Play with the API

You can actually click the button `Try it out` and send some requests to create some heroes with the **Create Hero** *path operation*.

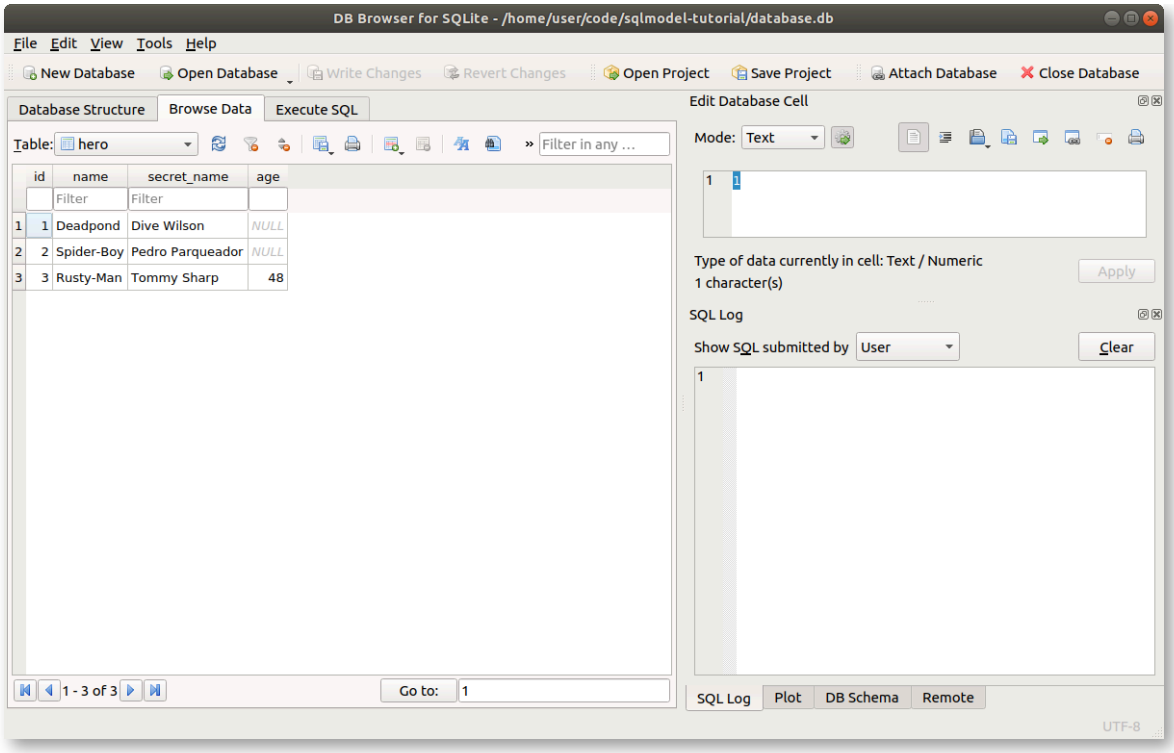
And then you can get them back with the **Read Heroes** *path operation*:



Check the Database

Now you can terminate that server program by going back to the terminal and pressing `Ctrl+C`.

And then, you can open **DB Browser for SQLite** and check the database, to explore the data and confirm that it indeed saved the heroes. 🎉



Recap

Good job! This is already a FastAPI **web API** application to interact with the heroes database. 🎉

There are several things we can improve and extend. For example, we want the database to decide the ID of each new hero, we don't want to allow a user to send it.

We will make all those improvements in the next chapters. 🚀